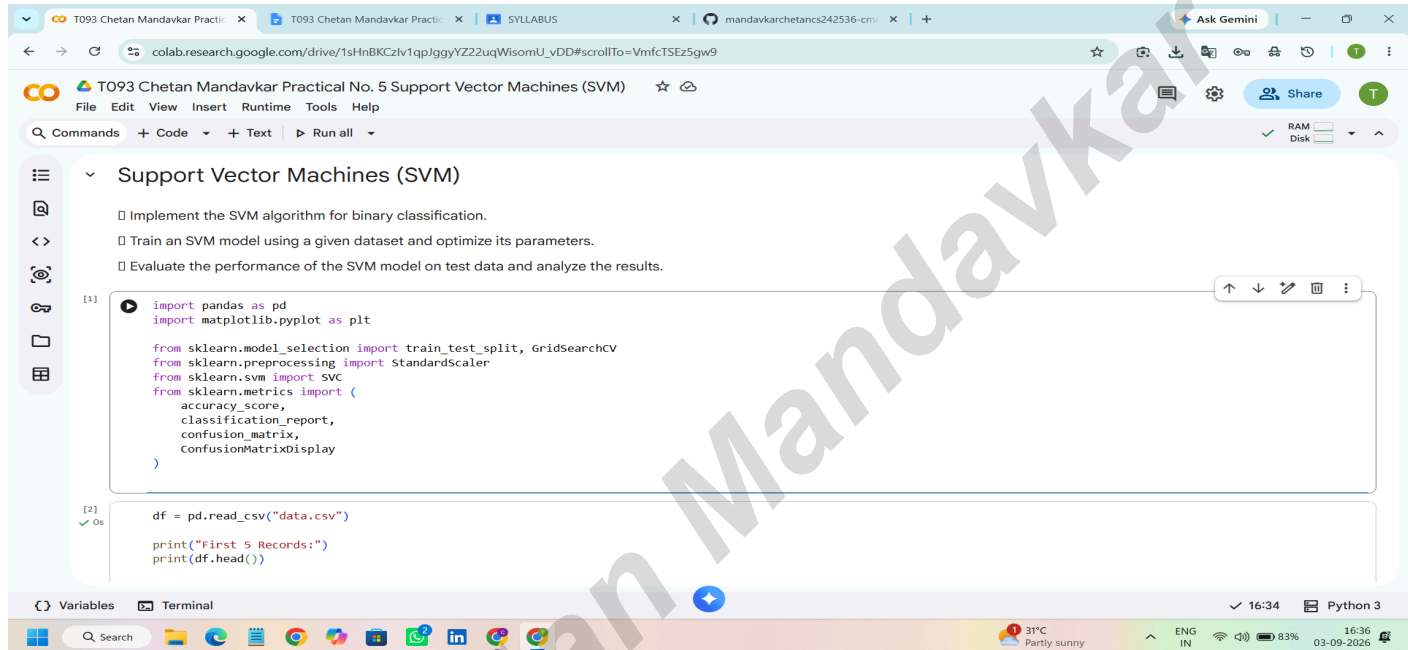


# Sheth L.U.J. And Sir M.V. College

Aim:- Support Vector Machines (SVM)

Implement the SVM algorithm for binary classification. Train an SVM model using a given dataset and optimize its parameters. Evaluate the performance of the SVM model on test data and analyze the results.



The screenshot shows a Google Colab notebook with the title "Support Vector Machines (SVM)". The notebook has a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu bar is a toolbar with "Commands", "Code", "Text", and "Run all". The notebook content is as follows:

```
[1] import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchcv
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    confusionMatrixDisplay
)

[2] df = pd.read_csv("data.csv")

print("First 5 Records:")
print(df.head())
```

The notebook interface includes a "Variables" panel on the left, a "Terminal" panel at the bottom, and a system tray at the bottom right showing the time as 16:34 and the date as 03-09-2026.



The screenshot shows the same Google Colab notebook, but now with the output of the code. The output for the first 5 records is displayed as follows:

|   | id       | diagnosis | radius_mean | texture_mean | perimeter_mean | area_mean |
|---|----------|-----------|-------------|--------------|----------------|-----------|
| 0 | 842302   | M         | 17.99       | 10.38        | 122.89         | 1801.0    |
| 1 | 842517   | M         | 20.57       | 17.77        | 132.90         | 1326.0    |
| 2 | 84300903 | M         | 19.69       | 21.25        | 130.00         | 1203.0    |
| 3 | 84348301 | M         | 11.42       | 20.38        | 77.58          | 386.1     |
| 4 | 84358402 | M         | 20.29       | 14.34        | 135.10         | 1297.0    |

The output for the dataset shape is also displayed:

```
print("\nDataset Shape:")
print(df.shape)
```

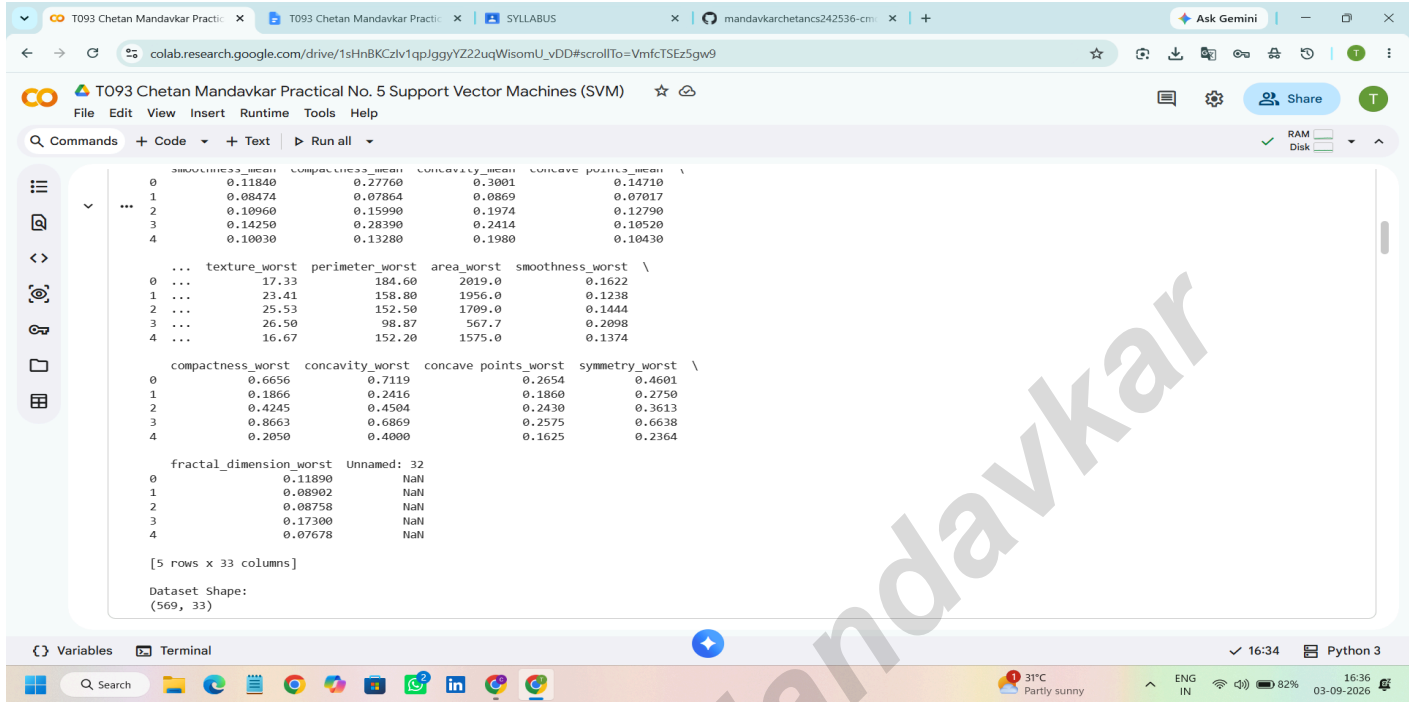
The notebook interface includes a "Variables" panel on the left, a "Terminal" panel at the bottom, and a system tray at the bottom right showing the time as 16:34 and the date as 03-09-2026.

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

# Sheth L.U.J. And Sir M.V. College



The screenshot shows a Google Colab environment with a Jupyter Notebook titled "T093 Chetan Mandavkar Practical No. 5 Support Vector Machines (SVM)". The notebook displays a preview of a dataset with 569 rows and 33 columns. The columns are grouped into several categories: compactness, texture, perimeter, area, smoothness, concavity, concave points, symmetry, and fractal dimension. The dataset shape is (569, 33).

```
compactness_mean compactness_worst compactness_se compactness_worst_mean \
0 0.11840 0.27760 0.3001 0.14710
1 0.08474 0.07864 0.0869 0.07017
2 0.10960 0.15990 0.1974 0.12790
3 0.14250 0.28390 0.2414 0.10520
4 0.10030 0.13280 0.1980 0.10430

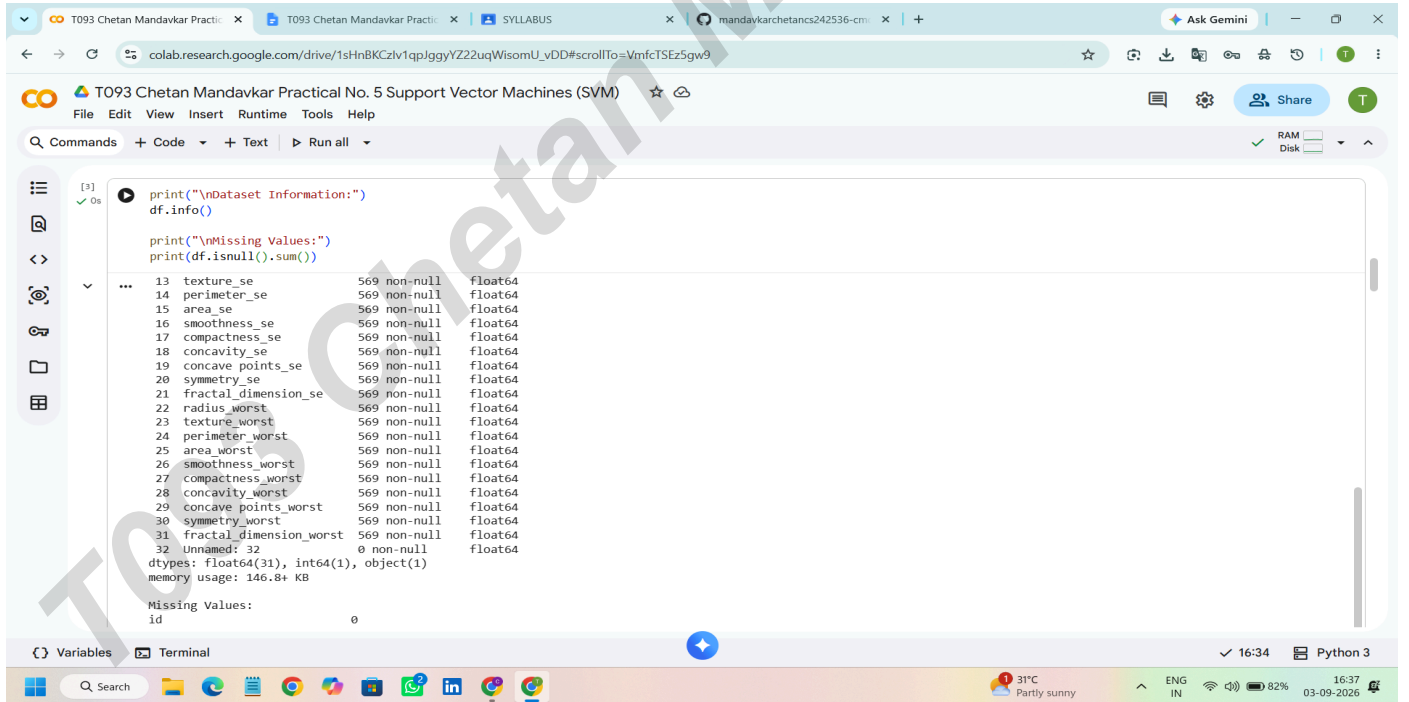
... texture_worst perimeter_worst area_worst smoothness_worst \
0 17.33 184.60 2019.0 0.1622
1 23.41 158.80 1956.0 0.1238
2 25.53 152.50 1709.0 0.1444
3 26.50 98.87 567.7 0.2098
4 16.67 152.20 1575.0 0.1374

compactness_worst concavity_worst concave points_worst symmetry_worst \
0 0.6656 0.7119 0.2654 0.4601
1 0.1866 0.2416 0.1860 0.2750
2 0.4245 0.4504 0.2430 0.3613
3 0.8663 0.6869 0.2575 0.6638
4 0.2050 0.4000 0.1625 0.2364

fractal_dimension_worst Unnamed: 32
0 0.11890 NaN
1 0.08902 NaN
2 0.08758 NaN
3 0.17300 NaN
4 0.07678 NaN

[5 rows x 33 columns]

Dataset Shape:
(569, 33)
```



The screenshot shows the same Google Colab environment, but now displaying the output of a code cell. The code prints dataset information, missing values, and dtypes. The output shows that the dataset has 569 non-null rows for each feature, and the dtypes are float64 for most features, int64 for 'id', and object for 'Unnamed: 32'. The memory usage is 146.8+ KB.

```
[3] print("\nDataset Information:")
df.info()

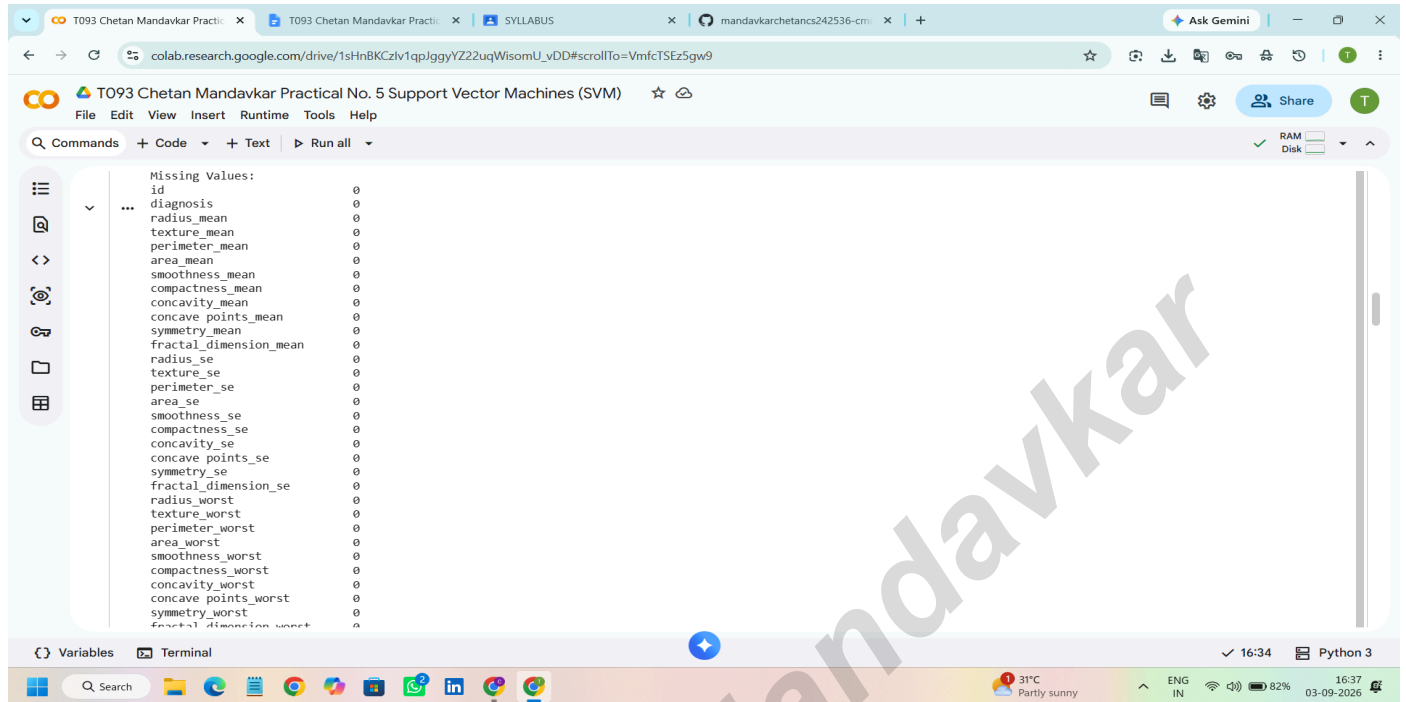
print("\nMissing Values:")
print(df.isnull().sum())

... 13 texture_se 569 non-null float64
14 perimeter_se 569 non-null float64
15 area_se 569 non-null float64
16 smoothness_se 569 non-null float64
17 compactness_se 569 non-null float64
18 concavity_se 569 non-null float64
19 concave points_se 569 non-null float64
20 symmetry_se 569 non-null float64
21 fractal_dimension_se 569 non-null float64
22 radius_worst 569 non-null float64
23 texture_worst 569 non-null float64
24 perimeter_worst 569 non-null float64
25 area_worst 569 non-null float64
26 smoothness_worst 569 non-null float64
27 compactness_worst 569 non-null float64
28 concavity_worst 569 non-null float64
29 concave points_worst 569 non-null float64
30 symmetry_worst 569 non-null float64
31 fractal_dimension_worst 569 non-null float64
32 Unnamed: 32 0 non-null float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB

Missing Values:
id 0
```

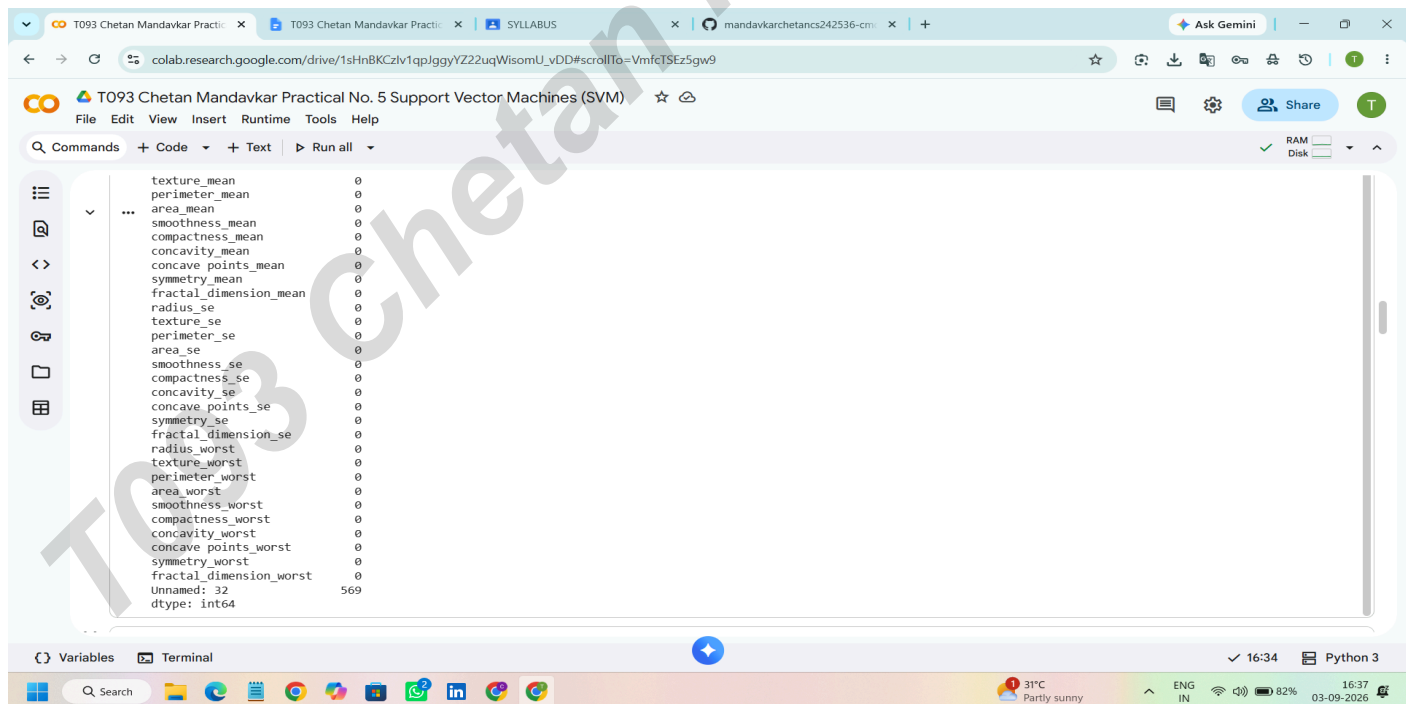
Name:- Chetan Mandavkar  
Roll No. T093  
Subject:- Artificial Intelligence

# Sheth L.U.J. And Sir M.V. College



The screenshot shows a Google Colab environment with a notebook titled "T093 Chetan Mandavkar Practical No. 5 Support Vector Machines (SVM)". The code cell displays a list of missing values for various features in a dataset. The features and their corresponding missing values are as follows:

| Feature                 | Missing Values |
|-------------------------|----------------|
| id                      | 0              |
| diagnosis               | 0              |
| radius_mean             | 0              |
| texture_mean            | 0              |
| perimeter_mean          | 0              |
| area_mean               | 0              |
| smoothness_mean         | 0              |
| compactness_mean        | 0              |
| concavity_mean          | 0              |
| concave points_mean     | 0              |
| symmetry_mean           | 0              |
| fractal_dimension_mean  | 0              |
| radius_se               | 0              |
| texture_se              | 0              |
| perimeter_se            | 0              |
| area_se                 | 0              |
| smoothness_se           | 0              |
| compactness_se          | 0              |
| concavity_se            | 0              |
| concave points_se       | 0              |
| symmetry_se             | 0              |
| fractal_dimension_se    | 0              |
| radius_worst            | 0              |
| texture_worst           | 0              |
| perimeter_worst         | 0              |
| area_worst              | 0              |
| smoothness_worst        | 0              |
| compactness_worst       | 0              |
| concavity_worst         | 0              |
| concave points_worst    | 0              |
| symmetry_worst          | 0              |
| fractal_dimension_worst | 0              |

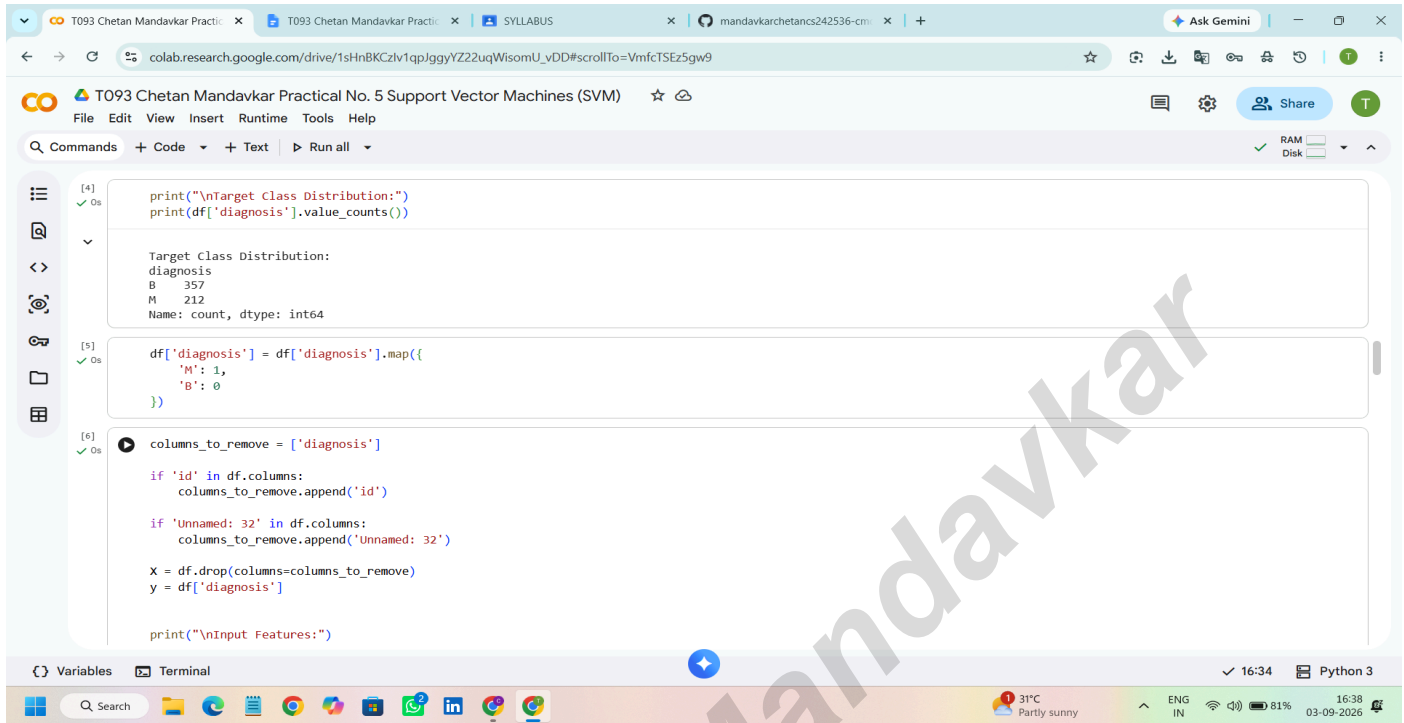


The screenshot shows the same Google Colab environment. The code cell displays the distribution of features in the dataset. The features and their corresponding distributions are as follows:

| Feature                 | Distribution |
|-------------------------|--------------|
| texture_mean            | 0            |
| perimeter_mean          | 0            |
| area_mean               | 0            |
| smoothness_mean         | 0            |
| compactness_mean        | 0            |
| concavity_mean          | 0            |
| concave points_mean     | 0            |
| symmetry_mean           | 0            |
| fractal_dimension_mean  | 0            |
| radius_se               | 0            |
| texture_se              | 0            |
| perimeter_se            | 0            |
| area_se                 | 0            |
| smoothness_se           | 0            |
| compactness_se          | 0            |
| concavity_se            | 0            |
| concave points_se       | 0            |
| symmetry_se             | 0            |
| fractal_dimension_se    | 0            |
| radius_worst            | 0            |
| texture_worst           | 0            |
| perimeter_worst         | 0            |
| area_worst              | 0            |
| smoothness_worst        | 0            |
| compactness_worst       | 0            |
| concavity_worst         | 0            |
| concave points_worst    | 0            |
| symmetry_worst          | 0            |
| fractal_dimension_worst | 0            |
| Unnamed: 32             | 569          |
| dtype:                  | int64        |

Name:- Chetan Mandavkar  
Roll No. T093  
Subject:- Artificial Intelligence

# Sheth L.U.J. And Sir M.V. College



```
[4] print("\nTarget Class Distribution:")
    print(df['diagnosis'].value_counts())

Target Class Distribution:
diagnosis
B    357
M    212
Name: count, dtype: int64

[5] df['diagnosis'] = df['diagnosis'].map({
    'M': 1,
    'B': 0
})

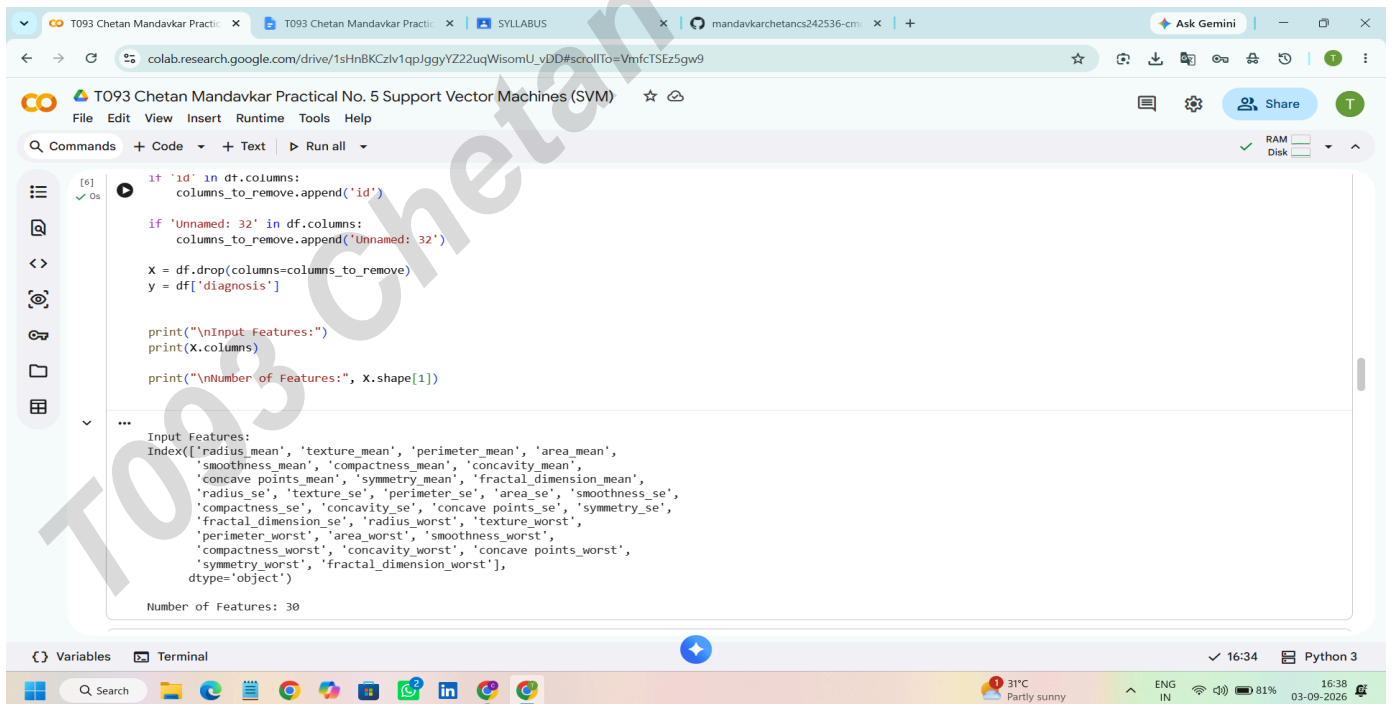
[6] columns_to_remove = ['diagnosis']

if 'id' in df.columns:
    columns_to_remove.append('id')

if 'Unnamed: 32' in df.columns:
    columns_to_remove.append('Unnamed: 32')

X = df.drop(columns=columns_to_remove)
y = df['diagnosis']

print("\nInput Features:")
```



```
[6] if 'id' in df.columns:
    columns_to_remove.append('id')

    if 'Unnamed: 32' in df.columns:
        columns_to_remove.append('Unnamed: 32')

    X = df.drop(columns=columns_to_remove)
    y = df['diagnosis']

    print("\nInput Features:")
    print(X.columns)

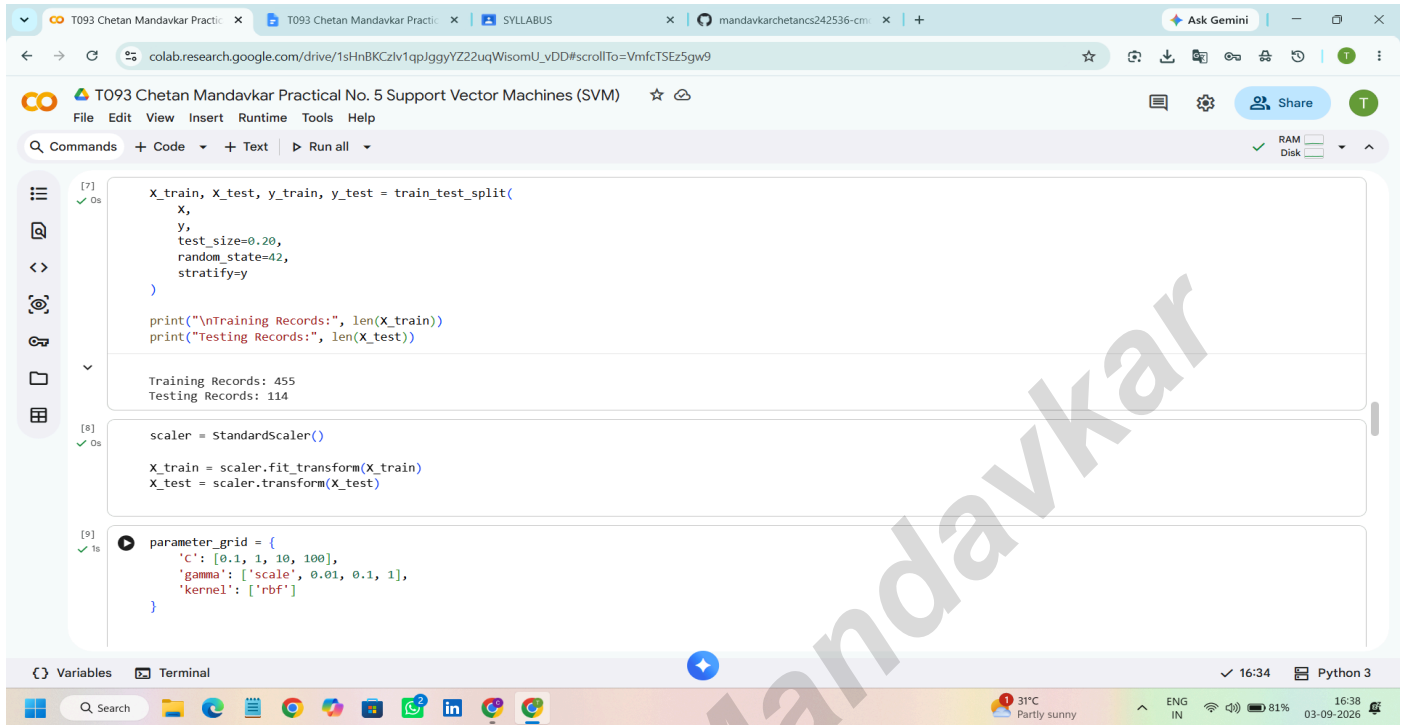
    print("\nNumber of Features:", X.shape[1])

...
Input Features:
Index(['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
       'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst'],
      dtype='object')

Number of Features: 30
```

Name:- Chetan Mandavkar  
Roll No. T093  
Subject:- Artificial Intelligence

# Sheth L.U.J. And Sir M.V. College



The screenshot shows a Google Colab notebook titled "T093 Chetan Mandavkar Practical No. 5 Support Vector Machines (SVM)". The notebook has three code cells. The first cell, labeled [7] 0s, contains the following code:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)

print("\nTraining Records:", len(X_train))
print("Testing Records:", len(X_test))
```

The output of this cell shows "Training Records: 455" and "Testing Records: 114". The second cell, labeled [8] 0s, contains the following code:

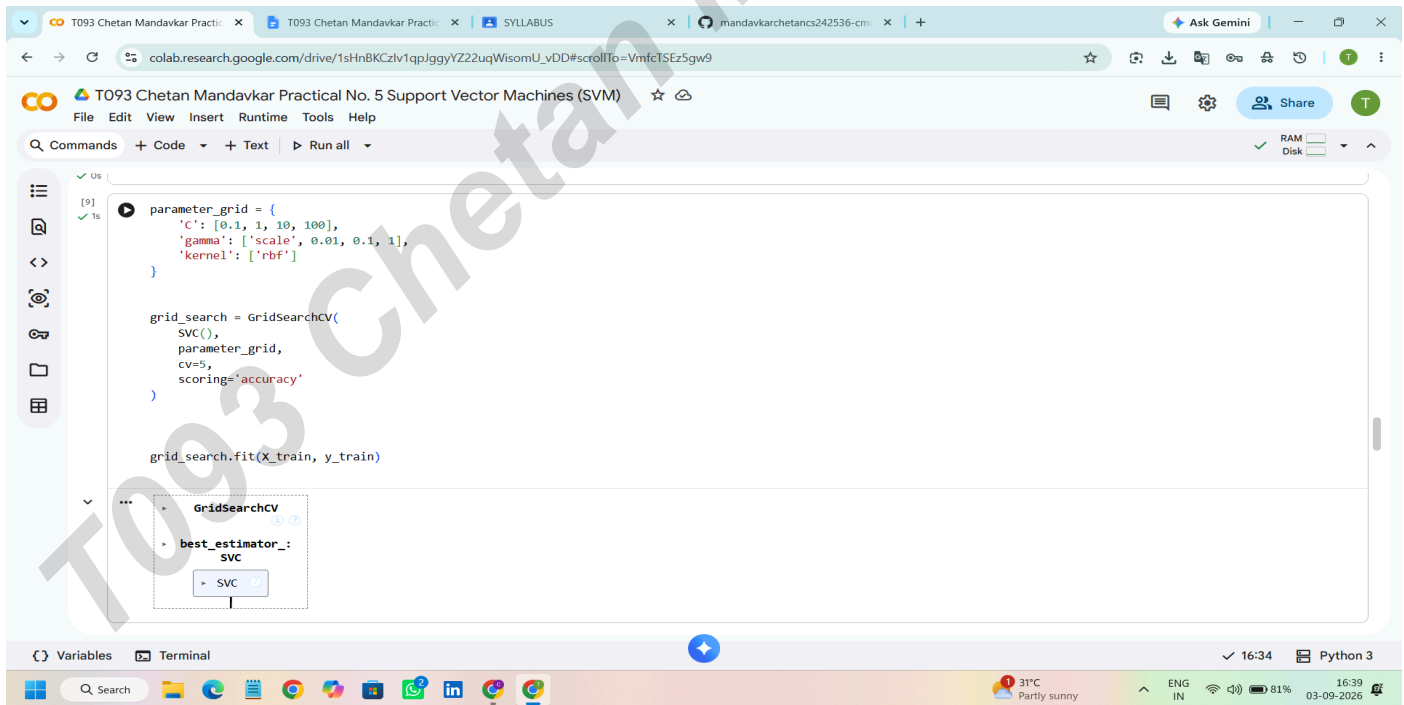
```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

The third cell, labeled [9] 1s, contains the following code:

```
parameter_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 0.01, 0.1, 1],
    'kernel': ['rbf']
}
```

The bottom of the notebook shows a terminal window and a status bar with the time 16:34 and Python 3.



The screenshot shows the same Google Colab notebook, but with the third code cell expanded to show the output of the GridSearchCV search. The code in the cell is:

```
parameter_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 0.01, 0.1, 1],
    'kernel': ['rbf']
}

grid_search = GridSearchCV(
    SVC(),
    parameter_grid,
    cv=5,
    scoring='accuracy'
)

grid_search.fit(X_train, y_train)
```

The output of the GridSearchCV search is shown in a box, indicating the best estimator is an SVC model.

Name:- Chetan Mandavkar  
Roll No. T093  
Subject:- Artificial Intelligence



```
[10] ✓ Os
print("\nBest Parameters:")
print(grid_search.best_params_)

print("\nBest Cross-Validation Accuracy:")
print(grid_search.best_score_ * 100, "%")

Best Parameters:
{'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}

Best Cross-Validation Accuracy:
97.58241758241759 %

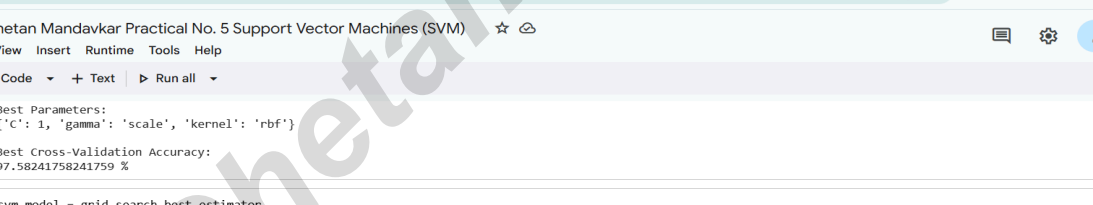
[11] ✓ Os
svm_model = grid_search.best_estimator_

[12] ✓ Os
y_pred = svm_model.predict(X_test)

print("\nActual Values:")
print(y_test.values)

print("\nPredicted Values:")
print(y_pred)

...
Actual Values:
[0 1 0 1 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 1 1 1 0 0 1 1 1 1 0 1 1 0 0 0
 0 1 0 0 0 1 0 0 1 0 1 0 1 1 0 0 0 1 0 0 0 1 0 0 0 1 1 0 1 0 1 1 0 0
 0 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```



```
Best Parameters:
{'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}

Best Cross-Validation Accuracy:
97.58241758241759 %

[11] ✓ 0s
svm_model = grid_search.best_estimator_

[12] ✓ 0s
y_pred = svm_model.predict(X_test)

print("\nActual Values:")
print(y_test.values)

print("\nPredicted Values:")
print(y_pred)

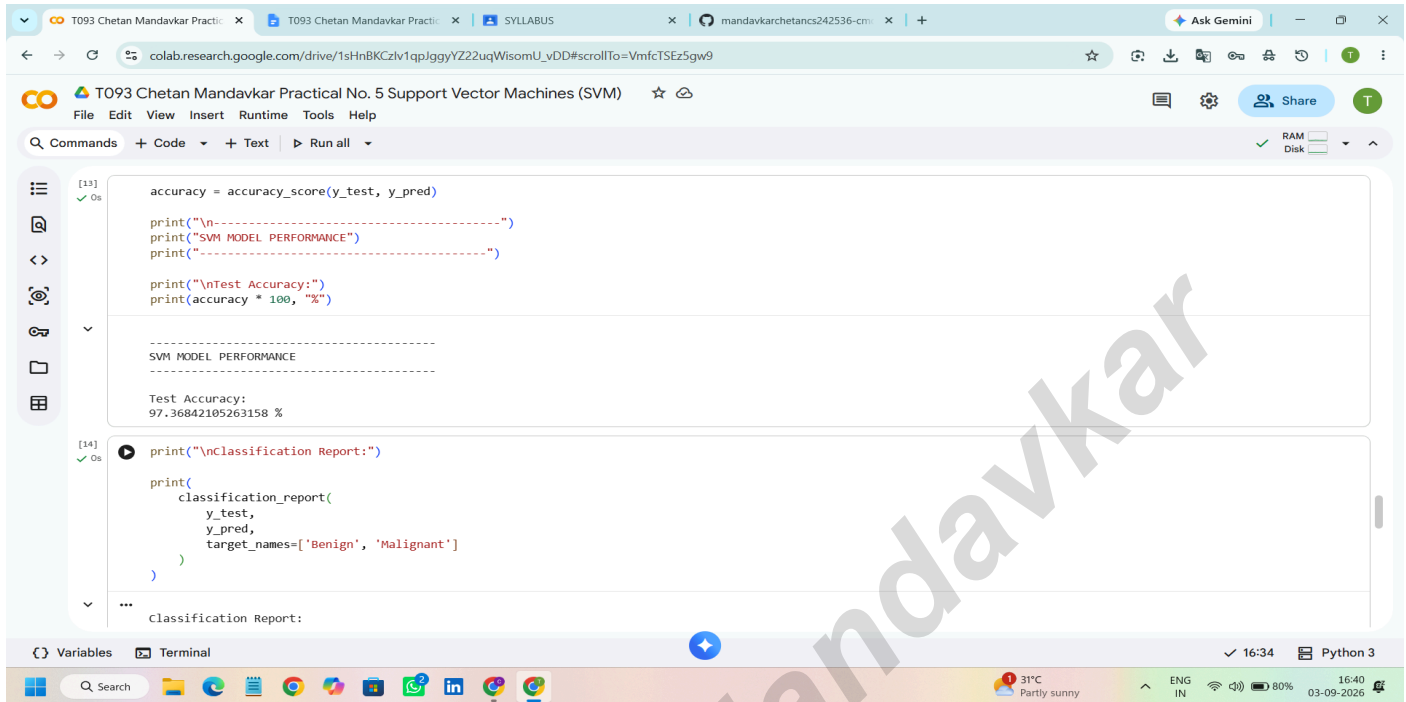
...

Actual Values:
[0 1 0 0 1 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 1 1 0 0 1 1 1 1 0 1 1 1 0 0 0
 0 1 0 0 0 1 0 0 1 0 1 0 0 1 1 0 0 0 0 1 0 0 0 1 0 0 0 1 1 0 1 0 1 1 1 0
 0 0 0 1 0 1 1 1 0 0 0 0 1 1 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 1 0 0
 0 1 1]

Predicted Values:
[0 1 0 0 0 0 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 1 0 1 1 1 0 0 0
 0 1 0 0 0 1 0 0 1 0 1 0 0 1 1 0 0 0 1 0 0 0 1 0 0 0 1 1 0 1 0 1 1 1 0
 0 0 1 0 1 1 1 1 0 0 0 0 1 1 1 0 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1 0
 0 1 1]
```

Name:- Chetan Mandavkar  
Roll No. T093  
Subject:- Artificial Intelligence

# Sheth L.U.J. And Sir M.V. College



The screenshot shows a Google Colab environment with the title "T093 Chetan Mandavkar Practical No. 5 Support Vector Machines (SVM)". The code cell [13] contains the following Python code:

```
accuracy = accuracy_score(y_test, y_pred)

print("\n-----")
print("SVM MODEL PERFORMANCE")
print("-----")

print("\nTest Accuracy:")
print(accuracy * 100, "%")
```

The output of this code is:

```
-----
SVM MODEL PERFORMANCE
-----
Test Accuracy:
97.36842105263158 %
```

The code cell [14] contains the following Python code:

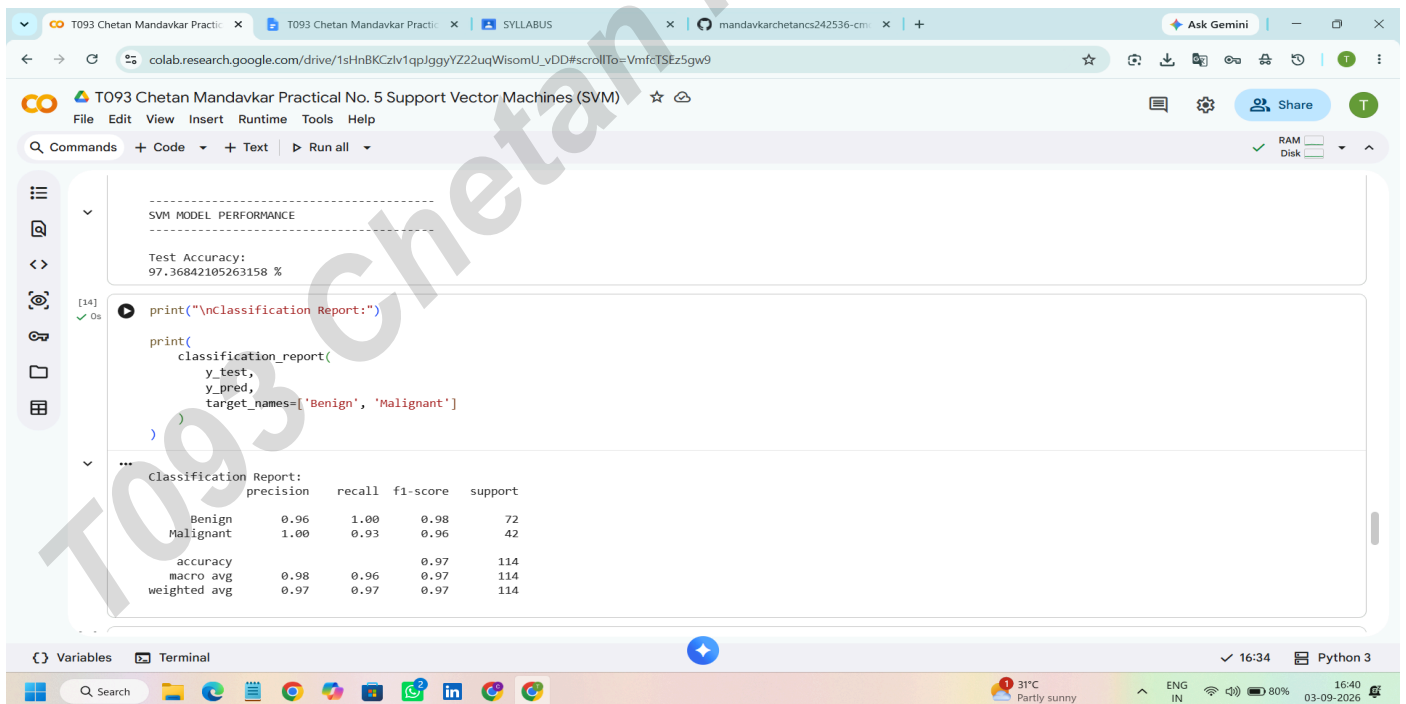
```
print("\nClassification Report:")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=['Benign', 'Malignant']
    )
)
```

The output of this code is:

```
Classification Report:
```

The bottom status bar shows the time as 16:34, the language as Python 3, and the date as 03-09-2026.



The screenshot shows the same Google Colab environment as the previous one. The code cell [14] is the same as in the previous screenshot. The output of this code is a detailed classification report:

```
Classification Report:
precision    recall  f1-score   support

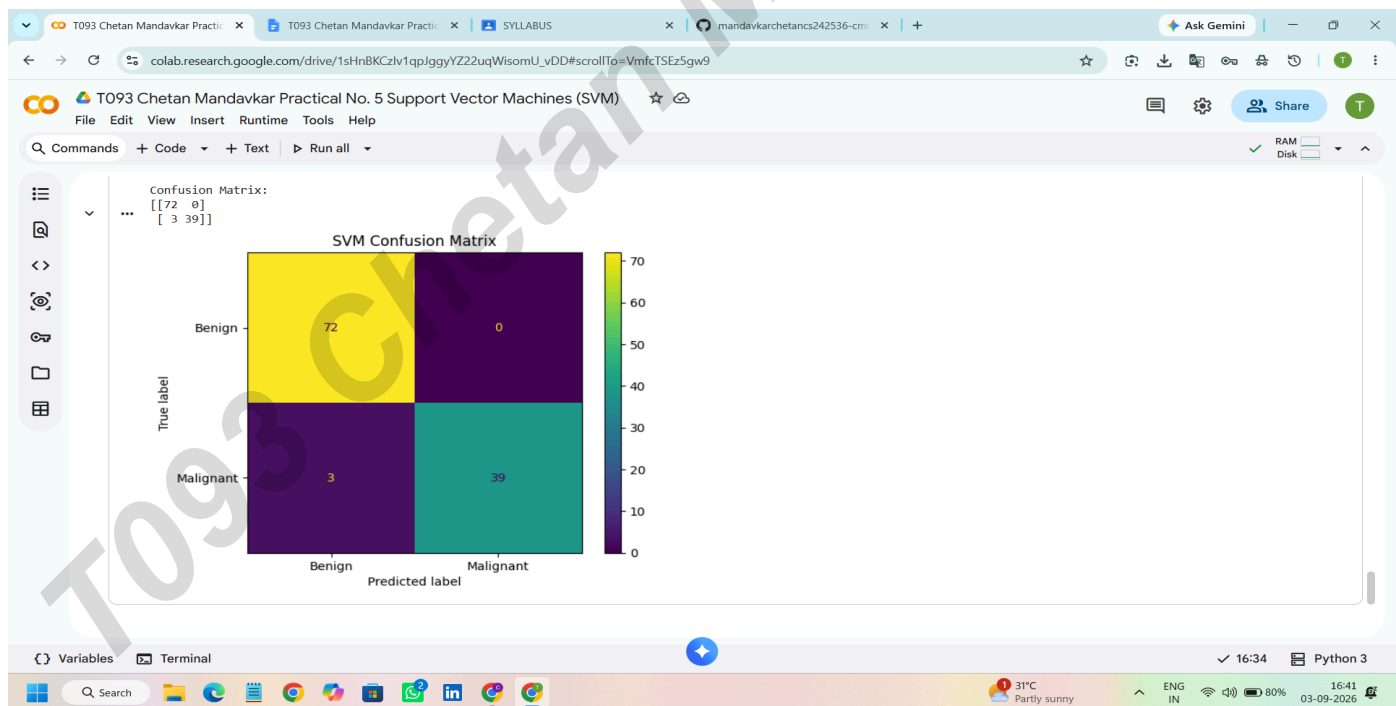
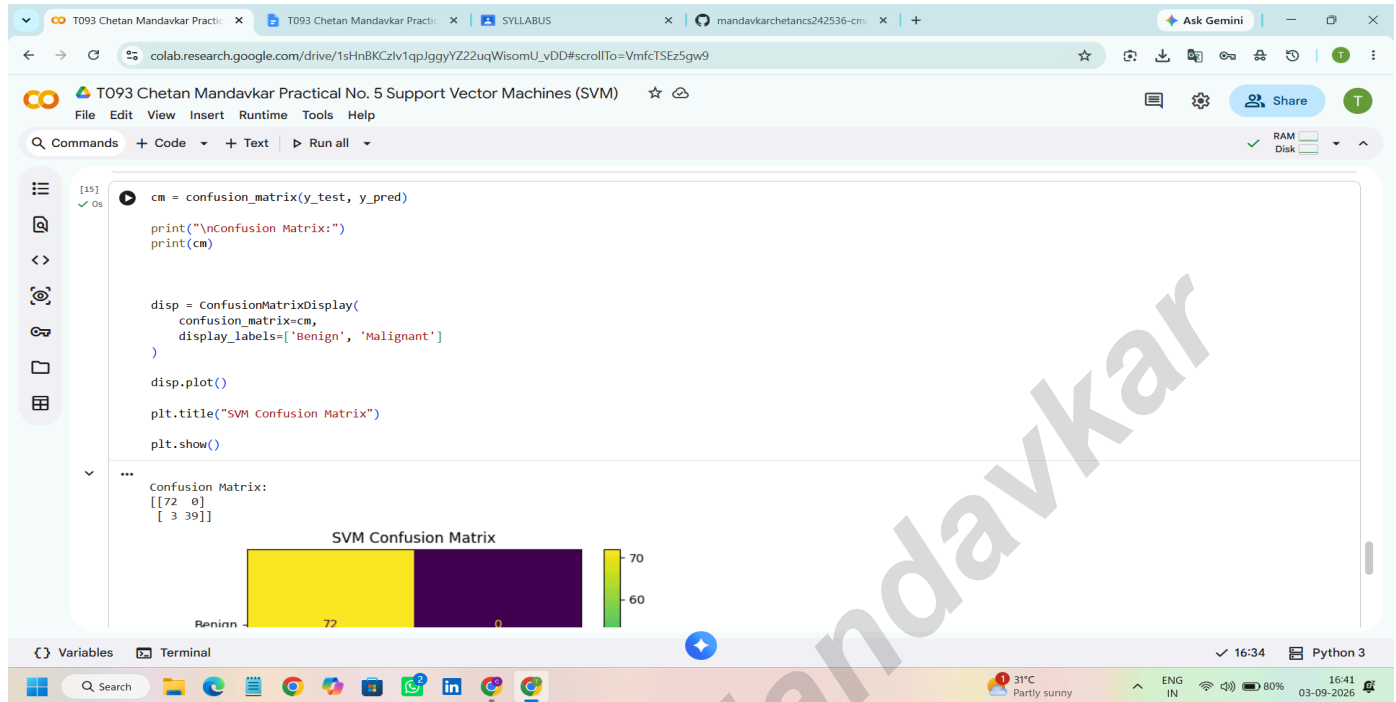
Benign       0.96     1.00     0.98        72
Malignant    1.00     0.93     0.96        42

accuracy     0.98     0.96     0.97       114
macro avg    0.98     0.96     0.97       114
weighted avg 0.97     0.97     0.97       114
```

The bottom status bar shows the time as 16:34, the language as Python 3, and the date as 03-09-2026.

Name:- Chetan Mandavkar  
Roll No. T093  
Subject:- Artificial Intelligence

# Sheth L.U.J. And Sir M.V. College



Name:- Chetan Mandavkar  
Roll No. T093  
Subject:- Artificial Intelligence